

Computational tasks of the layered-memory codes

a structured inventory: definitions, algorithms, scaling, speedups

July 2026 — working notes

0. The model and the central objects

All experiments use one estimator, the *layered mixture* of the companion paper. This section defines every object that the tasks below compute with; nothing later is used before it is defined here.

The prior, level by level. Fix an alphabet of size d . For a *level* (depth) $L \geq 1$, attach to each symbol x the weight

$$W_x = \prod_{\ell=1}^L E_{x,\ell}, \quad E_{x,\ell} \sim \text{Exp}(1) \text{ i.i.d.},$$

and let the random probability vector be $p_x = W_x / \sum_{y=1}^d W_y$. Larger L produces sparser, rougher vectors; L plays the role of a complexity (depth) parameter. The full estimator averages uniformly over $L = 1, \dots, L_{\max}$ with $L_{\max} = \lceil 2c^* \ln d \rceil$ ($c^* \approx 2.365$, the constant of the companion paper); measured $L_{\max} = 33\text{--}59$.

Profiles. A *row* is a vector of successor counts $(m_x)_{x=1}^d$, $m_x \geq 0$ (how often each next-symbol followed a given context). Its *profile* $\mathbf{m} = (m_1, \dots, m_s)$ is the multiset of the s *nonzero* counts, sorted; $N = \sum_i m_i$ is the row total. By exchangeability of the prior, all quantities below depend on a row only through its profile — the basis of all deduplication. We write $\tilde{k} \leq s$ for the number of *distinct* count values in $\mathbf{m}$, and μ_c for the multiplicity of the value c .

The profile probability. The central scalar is the probability that the level- L model assigns to (any one) sequence with profile $\mathbf{m}$:

$$q_L(\mathbf{m}) = \mathbb{E} \left[\prod_{x:m_x>0} p_x^{m_x} \right], \quad q_{\text{avg}}(\mathbf{m}) = \frac{1}{L_{\max}} \sum_{L=1}^{L_{\max}} q_L(\mathbf{m}). \quad (1)$$

Every codelength in the project is a sum of $-\log_2 q_{\text{avg}}$ terms over rows; every predictive probability is a ratio of two q_{avg} values.

The integral representation. The normalization $\sum_y W_y$ is removed by the Gamma identity $(\sum_y W_y)^{-N} = \frac{1}{\Gamma(N)} \int_0^\infty t^{N-1} e^{-t \sum_y W_y} dt$. Because the W_x are i.i.d., the expectation factorizes

under the integral, giving the exact formula

$$q_L(\mathbf{m}) = \frac{1}{\Gamma(N)} \int_0^\infty t^{N-1} \prod_{x=1}^d \varphi_{m_x}^{(L)}(t) dt, \quad \varphi_r^{(L)}(t) := \mathbb{E}[Y^r e^{-tY}], \quad Y = \prod_{\ell=1}^L E_\ell. \quad (2)$$

Here $\varphi_r^{(L)}$ — the *moment function* — is the r -th moment of a product of L independent unit exponentials, damped by e^{-tY} ; $t > 0$ is a *scalar* integration variable. The $d - s$ symbols with $m_x = 0$ contribute the common factor $\varphi_0^{(L)}(t)^{d-s}$. Numerically everything lives on the logarithmic axis $u = \ln t$ (a scalar), on a finite grid $u_1 < \dots < u_G$ spanning $[u_{\min}, u_{\max}]$ (measured $G = 3.6 \cdot 10^4$ at the largest settings), and in log space: with the *log-integrand*

$$\psi_L(u) = Nu + (d - s) \ln \varphi_0^{(L)}(e^u) + \sum_{\text{distinct } c \in \mathbf{m}} \mu_c \ln \varphi_c^{(L)}(e^u), \quad (3)$$

one has $q_L(\mathbf{m}) = \frac{1}{\Gamma(N)} \int e^{\psi_L(u)} du$ (the substitution $t = e^u$ contributes the extra factor t that turns t^{N-1} into e^{Nu}).

Two exact boundary facts. (i) $\varphi_r^{(1)}(t) = \Gamma(r+1)(1+t)^{-(r+1)}$ (closed form, the recursion's base). (ii) As $t \rightarrow 0$, $\varphi_r^{(L)}(t) \rightarrow \Gamma(r+1)^L$; hence to the left of the grid $\psi_L(u) = Nu + \text{const} + O(e^u)$ and the left tail of the integral is available in closed form. For $L = 1$ the whole integral is closed-form (a Dirichlet–multinomial); no tables are needed.

The tasks, in dependency order. Symbols and measured magnitudes on `text8` ($n = 1.7 \cdot 10^7$ tokens):

sym.	meaning	measured	sym.	meaning	measured
n	corpus length (tokens)	$1.7 \cdot 10^7$	N	row total $\sum_i m_i$	$1 - 10^6$
d	alphabet of a predictor	$256 - 2.6 \cdot 10^5$	s	nonzero successors of a row	$1 - 4,096$
$L_{\max}$	levels, $\lceil 2c^* \ln d \rceil$	$33 - 59$	$\tilde{k}$	distinct count values of a row	$1 - \sim 300$
G	grid points on the u -axis	$3.6 \cdot 10^4$	$\mathcal{N}$	rows (context states) of a model	$10^3 - 2.5 \cdot 10^5$
Q	Gauss–Laguerre order (T1)	96	P	distinct (context, succ.) pairs	up to $1.5 \cdot 10^7$
$\mathcal{R}$	set of count values needed	$10^3 - 5 \cdot 10^3$	U	unique profiles after dedup	$2.6 - 7 \cdot 10^4$
C	refreshes of a sequential code	$8 - 32$	D	pooled lag experts	6
K	family members (weight grid)	$\sim 10^2$			

1 T1 — Building the moment tables

Task. Tabulate the moment functions $\varphi_r^{(L)}$ of Eq. (2) on the u -grid, for every level $L \leq L_{\max}$ and every count value r that occurs in the data. Built once per alphabet size, cached on disk, read by all later tasks.

Formula. Conditioning on the last factor $E_L = e$ of $Y = Y'E_L$ gives the recursion in the level:

$$\varphi_r^{(\ell)}(t) = \mathbb{E}[E^r \varphi_r^{(\ell-1)}(tE)] = \int_0^\infty e^{-e} e^r \varphi_r^{(\ell-1)}(te) de, \quad \varphi_r^{(1)}(t) = \frac{\Gamma(r+1)}{(1+t)^{r+1}}. \quad (4)$$

Input/Output. In: the set $\mathcal{R}$ of needed count values (from the data: every value occurring in any profile), $L_{\max}$, the grid $\{u_g\}_{g=1}^G$. Out: the array $\ln \varphi_r^{(L)}(e^{u_g}) \in \mathbb{R}^{|\mathcal{R}| \times L_{\max} \times G}$, stored on disk (one file per r).

Algorithm. For each r independently: start from the closed form at $\ell = 1$; for $\ell = 2, \dots, L_{\max}$ evaluate Eq. (4) at every grid point by Gauss–Laguerre quadrature of order Q , reading $\varphi_r^{(\ell-1)}$ at the shifted points $u_g + \ln e_j$ by linear interpolation in u (with the exact left fill $\Gamma(r+1)^{\ell-1}$ beyond the grid). Embarrassingly parallel over r ; new r values are built incrementally as the data produces them.

Cost and scaling. Build: $O(|\mathcal{R}| \cdot L_{\max} \cdot G \cdot Q)$ operations ($\approx 10^{12}$ at the largest settings; measured 45 minutes on 64 cores). Storage: $8|\mathcal{R}|L_{\max}G$ bytes = 85 GB there. $|\mathcal{R}|$ grows slowly with corpus size (new count values become rare); G must grow like the *span* of the u -axis, i.e. with $\ln(\max r)$ and $L_{\max}$; $L_{\max} \propto \ln d$. The recurring cost is *reading*: T2/T3 gather $\tilde{k}$ table rows per profile per level, so layout and locality dominate in practice (network filesystems: measured failure mode).

Speedup routes. (1) [exact] One shared cache across all experiments (measured waste: 219 GB of duplicate tables). (2) [exact] Level-major single-file layout + float32: $4\times$ smaller, one file open per level instead of $|\mathcal{R}|$. (3) [approx] Large- r asymptotics: for $r \gg 1$, $\varphi_r^{(L)}$ concentrates (the integrand of Eq. (4) peaks sharply) and approaches a scaling form in (r, tr^L) ; replacing tables beyond a threshold r^* by the asymptotic formula shrinks $\mathcal{R}$ to the small counts where tables are genuinely needed. Needs a worked-out error bound before use.

2 T2 — Evaluating a profile probability (the atom)

Task. Compute $\log q_L(\mathbf{m})$ for all levels L , hence $\log q_{\text{avg}}(\mathbf{m})$, for one profile; executed once per unique profile per model (static runs) and once per refresh per active row (sequential runs).

Formula. Eqs. (1)–(3): $\log q_L = \log \int e^{\psi_L(u)} du - \ln \Gamma(N)$.

Input/Output. In: profile $\mathbf{m}$ as pairs (distinct value c , multiplicity μ_c); the tables (T1). Out: the vector $(\log_2 q_L(\mathbf{m}))_{L=1}^{L_{\max}}$ and $\log_2 q_{\text{avg}}$.

Algorithm. Per level: (i) assemble ψ_L on the grid by Eq. (3) — $\tilde{k} + 1$ table rows, $O(G\tilde{k})$; (ii) locate every local maximum of ψ_L within a significance window of the global one (the integrand can be multimodal: deep levels with heavy counts produce a second, far-left peak); (iii) refine each peak by a bracketed saddle solve (the derivative $\psi'_L(u)$ is available from neighboring table rows) and add Laplace contributions by log-sum-exp; (iv) add the closed-form left tail of Section 0. Special cases short-circuit: $N \leq 1$ by symmetry, $L = 1$ closed form. Batched over profiles; parallelized over profile chunks, each level’s table rows distributed to workers through shared memory.

Cost and scaling. Dominated by step (i): $O(L_{\max} G \tilde{k})$ per profile — from $4 \cdot 10^6$ ops ($\tilde{k} = 2$) to $2 \cdot 10^8$ ($\tilde{k} \sim 100$). Measured: $7 \cdot 10^4$ unique profiles at $L_{\max} = 59$ in 17 minutes on 64 cores. Static models invoke T2 only U times (deduplication); sequential models invoke it per refresh (T8).

Speedup routes. (1) [exact] *Warm-started peaks / local grids*: the full-grid sweep exists only to locate peaks robustly. Peak locations vary smoothly with the profile (and, across refreshes of the same row, barely move); warm-starting the saddle solve from a stored or predicted location and evaluating ψ_L only on a local window around it replaces G by $G_{\text{loc}} \sim 10^2$: up to $300\times$ on step (i), with a cheap global-sweep fallback when the bracketed solve fails or the result looks suspicious. (2) [approx] *Level truncation with certified bounds*: the L -sum of Eq. (1) is dominated by a few levels around the maximizing depth, with geometric decay away from it; exact evaluation of the top levels plus a tail bound replaces $L_{\max}$ by $L_{\text{eff}} \sim 5\text{--}10$. (3) [approx] Coarser grids for small- N profiles (their integrands are wide and smooth); per-profile grid selection by target accuracy.

3 T3 — The predictive table of a row

Task. Turn one row’s counts into a next-symbol probability distribution — the workhorse of sequential (pooled) codes, executed at every refresh for every row of every expert.

Formula. For a symbol whose current count in the row is c (including $c = 0$ for the unseen symbols, when $s < d$):

$$p(\text{symbol} \mid \text{row}) = \frac{q_{\text{avg}}(\mathbf{m} \oplus c)}{q_{\text{avg}}(\mathbf{m})}, \quad \mathbf{m} \oplus c := \mathbf{m} \text{ with one count } c \rightarrow c + 1 \quad (5)$$

(exchangeability again: the value depends on the symbol only through c , so the table has $\tilde{k} + 1$ distinct entries; the entries sum to one exactly, and are renormalized to absorb quadrature error, measured $\sim 5 \cdot 10^{-5}$ relative).

Input/Output. In: the row’s counts; tables (T1). Out: $\log_2 p$ over the alphabet ($\tilde{k} + 1$ distinct values).

Algorithm. Today: one T2 evaluation for $\mathbf{m}$ and one for each of the $\tilde{k} + 1$ augmented profiles $\mathbf{m} \oplus c$ — i.e. $\tilde{k} + 2$ independent atom evaluations per row, memoized by profile.

Cost and scaling. $O(L_{\max} G \tilde{k}^2)$ per row. Measured workload: $5.9 \cdot 10^6$ atom evaluations for the $V = 1,024$, $C = 32$, six-lag pooled run — the dominant cost of sequential codes, and memoization is weak because active rows change between refreshes.

Speedup routes. (1) [exact; the largest single lever] *Shared integrand*: by Eq. (3), every augmented profile differs from the base in ONE factor:

$$q_L(\mathbf{m} \oplus c) = \frac{1}{\Gamma(N+1)} \int e^{\psi_L(u)} e^u \frac{\varphi_{c+1}^{(L)}(e^u)}{\varphi_c^{(L)}(e^u)} du.$$

Assemble ψ_L once ($O(G\tilde{k})$); each predictive entry is then a single $O(G)$ weighted sum against a ratio of two table rows. Per-row cost falls from $O(G\tilde{k}^2)$ to $O(G\tilde{k})$: a factor $\tilde{k}$ ($\sim 100\times$ exactly on the rich rows that dominate), with bit-identical output up to quadrature details. (2) [exact] *Incremental integrand across refreshes*: between refreshes few counts of a row change; update $\psi'_L = \psi_L + \sum_{\text{changed}} (\ln \varphi_{c'}^{(L)} - \ln \varphi_c^{(L)})$ in $O(G \cdot \#\text{changed})$ instead of reassembling. Requires storing ψ_L for active rows ($G \times L_{\max}$ float32 ≈ 8 MB per row at the largest settings — affordable exactly for the expensive frequent rows). (3) T2's routes apply on top (warm peaks, level truncation).

4 T4 — Counting: rows, profiles, deduplication

Task. Extract from the token stream, for a given state map, the successor-count rows, their profiles, and the row-to-unique-profile map.

Formula. $\text{count}[s, x] = \#\{t : \sigma(x^{t-1}) = s, x_t = x\}$ for a state map σ (last token, product state, tree context, or lag- δ token); $\text{profile}(s)$ as in Section 0; the dedup map $s \mapsto \text{profile id}$.

Input/Output. In: token ids (n integers), σ . Out: $\mathcal{N}$ rows (P nonzero entries), U unique profiles, dedup map.

Algorithm. One pass with hash-map counting; per-row sort of nonzero counts; profile hashing.

Cost and scaling. Time $O(n)$ per state map — minutes, never the bottleneck. *Memory* $O(P)$ with a dictionary constant of 30–60 bytes per entry: measured ≈ 200 GB at $P \approx 1.5 \cdot 10^7$ (full vocabulary, depth 2) — this is what forbids depth 3 today. Deduplication is the great equalizer downstream: $P = 4.4 \cdot 10^6$ contexts collapse to $U = 7 \cdot 10^4$ profiles.

Speedup routes. (1) [exact] External sort-and-merge counting: $O(n \log n)$ on disk, constant memory; standard technology. (2) [exact] Suffix-array counting: one compact $O(n)$ -memory index of the token stream yields the counts of *all* contexts of *all* depths (each context is an interval of the suffix array); the natural route to depth ≥ 3 at full vocabulary. (3) [exact] Fixed-width integer arrays instead of dictionary objects.

5 T5 — Aggregation: families, trees, posteriors

Task. Combine per-row atoms into model codelengths, mixtures over model families, and posteriors.

Formula. Member codelength $\sum_{\text{rows } s} -\log_2 q_{\text{avg}}(\mathbf{m}_s)$; the context-tree mixture over all prunings of the depth- D context trie by the recursion

$$\beta(v) = \frac{1}{2} q_{\text{avg}}(\mathbf{m}_v) + \frac{1}{2} \prod_{c \in \text{children}(v)} \beta(c)$$

(bottom-up over observed contexts v); a family mixture over K members = logsumexp of member log-likelihood totals minus $\log_2 K$; the posterior is the normalized exponential of the same totals.

Input/Output/Algorithm/Cost. In: per-profile $\log_2 q_{\text{avg}}$ (T2), trie structure. Out: bits/token per member, family bits, posterior. Single passes, $O(\mathcal{N})$, $O(P)$, $O(K)$; measured seconds. No scaling concern.

6 T6 — The additive pooling pass

Task. Sequentially code the corpus under every additive (mixture) weight vector of the grid simultaneously.

Formula. With experts $e = 0$ (memoryless) and $e = 1..D$ ($\log \delta_e$), per-step probabilities $p_e(x_t)$ read from the T3 tables of the current refresh window, and weight vectors $\lambda \in \Lambda$, $|\Lambda| = K$:

$$\text{bits}(\lambda) = \sum_t -\log_2 \sum_{e=0}^D \lambda_e p_e(x_t).$$

Input/Output. In: the $n \times (D+1)$ matrix of realized-token probabilities (one gather per expert per step); Λ . Out: K codelengths.

Algorithm/Cost. Blocked gather, one $(\text{block} \times (D+1)) \cdot ((D+1) \times K)$ matrix product per block, log, accumulate: $O(n(D+1)K) \approx 10^{10}$ flops — minutes. Linear in everything; never a concern.

7 T7 — The multiplicative pooling pass

Task. Same as T6 for the tempered-product rule, which must renormalize over the alphabet at *every* position.

Formula. With exponents $\beta \in B$, $|B| = K_{\text{prod}}$:

$$p_\beta(x_t) = \frac{s_\beta(x_t)}{Z_t}, \quad s_\beta(x) = q_0(x) \prod_{j=1}^D \left(\frac{p_j(x)}{q_0(x)} \right)^{\beta_j}, \quad Z_t = \sum_{x=1}^V s_\beta(x),$$

where q_0 is the memoryless expert’s T3 table.

Input/Output. In: T3 tables of the D experts and q_0 ; B . Out: K_{prod} codelengths.

Algorithm (today) and cost. Dense: per block, gather each expert’s full table rows for the occurring contexts, form the V -vector of logits per step per member, log-sum-exp. $O(n V D K_{\text{prod}})$: measured 68 minutes at $V = 4,096$; $\sim 10^{16}$ at $V = 10^6$, $n = 1.5 \cdot 10^8$. **The only truly bad exponent in the project.**

Speedup route. [exact; kills the V factor] By Eq. (5) every T3 table is “one background value plus s_j exceptions”: $p_j(x) = a_j$ for every x unseen in row j ’s counts, individual values only on the s_j observed successors. Let $S_t = \bigcup_j \text{supp}_j(t)$ (union of the exception sets; $|S_t| \sim 10^2\text{--}10^3 \ll V$). For $x \notin S_t$ the product collapses to a constant times $q_0(x)^{1-\sum_j \beta_j}$, so

$$Z_t = \left(\prod_j a_{j,t}^{\beta_j} \right) \left(T_\gamma - \sum_{x \in S_t} q_0(x)^\gamma \right) + \sum_{x \in S_t} s_\beta(x), \quad \gamma = 1 - \sum_j \beta_j, \quad T_\gamma = \sum_{x=1}^V q_0(x)^\gamma,$$

with the power sums T_γ precomputed once per refresh per needed γ . Exact Z_t in $O(|S_t|D)$ per step per member instead of $O(VD)$.

8 T8 — Refresh scheduling of sequential codes

Task. Decide when each expert row’s predictive table (T3) is rebuilt. Any schedule that depends only on the past yields a valid sequential code; staleness only lengthens it. This schedule is the multiplier on T3 that separates static runs (minutes) from pooled runs (hours).

Formula. Current schedule: a global clock; all rows rebuilt at C equally spaced positions; T3 invocations = $C \times (\text{active rows})$.

Input/Output. In: running counts. Out: refresh times per row; the tables used between refreshes.

Cost and scaling. T3 cost $\times C$, with memoization weakened by row churn (measured: $5.9 \cdot 10^6$ atom evaluations at $V = 1,024$, $C = 32$).

Speedup routes. (1) [valid code; controlled staleness] *Growth-triggered refreshes*: rebuild a row when its total N has grown by a factor $1 + \epsilon$ since its last rebuild. Refreshes per row drop from C to $O(\log_{1+\epsilon} N_{\text{final}})$, staleness is bounded *relatively* (the quantity that matters for predictive quality), and the trigger is past-measurable, so the code remains valid. Combined with T3’s shared/incremental integrands the compound gain on pooled runs is estimated at 10^2 – 10^3 . (2) [exact] Shard rows across cluster nodes (rows are independent given the schedule).

Scaling summary and discussion priority

At $10\times$ the corpus (enwik9-scale: $n \approx 1.4 \cdot 10^8$, $V \sim 10^6$) with today’s algorithms: T4 needs terabytes of RAM (blocked); T3 $\times$ T8 explodes (richer rows, C multiplier: weeks); T7 dense is impossible; T1/T2 grow only through $|\mathcal{R}|$, G , L_{max} (manageable). With the [exact] routes — shared and incremental integrands (T3), warm-started peaks (T2), sparse normalization (T7), growth-triggered refreshes (T8), external or suffix-array counting (T4), sharding — every task returns to node-hours; the [approx] routes (level truncation, large- r asymptotics, coarse grids) are held in reserve, each with a checkable error.

Suggested discussion order: T3(1) shared integrand; T7 sparse normalizer; T8 growth-triggered refreshes; T2(1) warm-started peaks; T4 suffix arrays; then the approximations if still needed.